

Lời giải chi tiết bài tập nhóm 4

Phân tích và thiết kế thuật toán

Nguyễn Duy Khang - 24520755

Hà Bùi Trọng Nghĩa - 24520020

Ngày 20 tháng 11 năm 2025

1. Mô tả bài toán

Cho một bản đồ hình chữ nhật từ $(0, 0)$ đến (W, H) . Trên bản đồ có:

- n vùng theo dõi Aura, mỗi vùng là hình chữ nhật xác định bởi tọa độ góc trái dưới (X_i, Y_i) và góc phải trên (A_i, B_i) . Các vùng không giao nhau, không tiếp xúc biên bản đồ.
- m Hunter, mỗi Hunter tại tọa độ (C_j, R_j) với lượng Aura t_j .

Nhiệm vụ: Tính tổng lượng Aura được ghi nhận trong từng vùng. Hunter được ghi nhận vào vùng nhỏ nhất chứa vị trí của họ; nếu Hunter không thuộc vùng nào, lượng Aura được cộng vào phần “không theo dõi”.

Đầu vào:

- Dòng đầu: W, H, n, m
- n dòng tiếp theo: X_i, Y_i, A_i, B_i
- m dòng tiếp: C_j, R_j, t_j

Đầu ra: $n + 1$ số nguyên, số thứ i là tổng Aura của vùng thứ i , số cuối cùng là tổng Aura không theo dõi.

2. Phân tích bài toán

2.1 Đặc điểm quan trọng

- Các vùng không giao nhau, không tiếp xúc biên.

- Hunter có thể nằm trong hoặc ngoài vùng.
- Subtask khác nhau dựa trên kích thước dữ liệu (n, m, W, H) .

2.2 Yêu cầu thuật toán

- Phân bổ đúng lượng Aura cho từng vùng.
- Đảm bảo hiệu quả với dữ liệu lớn.
- Có thể áp dụng nhiều phương pháp tùy subtask.

3. Solution chi tiết từng Subtask

3.1 Subtask 1: $n, m \leq 10^3$

Hướng tiếp cận:

Với giới hạn nhỏ $n, m \leq 1000$, ta có thể kiểm tra trực tiếp mỗi Hunter với từng vùng:

- Với mỗi Hunter, duyệt toàn bộ danh sách các vùng.
- Kiểm tra Hunter có nằm trong vùng hay không bằng điều kiện:

$$X_i \leq C_j < A_i \quad \text{và} \quad Y_i \leq R_j < B_i$$

- Nếu Hunter nằm trong nhiều vùng, chọn vùng có diện tích nhỏ nhất.
- Nếu Hunter không thuộc bất kỳ vùng nào, cộng Aura vào phần “không theo dõi”.

Phương pháp thuật toán:

- Brute Force (kiểm tra tất cả các cặp Hunter - vùng).
- Đây là phương pháp phù hợp khi dữ liệu nhỏ vì không cần cấu trúc dữ liệu phức tạp.

Lý do chọn phương pháp:

- Số phép so sánh tối đa là $n \times m \leq 10^6$, đảm bảo chạy nhanh trong thời gian thực thi.
- Triển khai dễ dàng, dễ kiểm soát và ít lỗi.

Phân tích độ phức tạp:

- Thời gian: $O(n \cdot m)$
- Bộ nhớ: $O(n + m)$

Code Python:

```
1 def subtask1(n, m, regions, hunters):
2     ans = [0] * (n + 1)
3     areas = [(A - X) * (B - Y) for X, Y, A, B in regions]
4
5     for cx, cy, t in hunters:
6         candidate = -1
7         min_area = 10**18
8         for i, (X, Y, A, B) in enumerate(regions):
9             if X <= cx < A and Y <= cy < B:
10                 if areas[i] < min_area:
11                     min_area = areas[i]
12                     candidate = i
13         if candidate == -1:
14             ans[n] += t
15         else:
16             ans[candidate] += t
17     return ans
```

3.2 Subtask 2: $W, H \leq 10^3$

Hướng tiếp cận:

- Dữ liệu bản đồ nhỏ ($W, H \leq 1000$), ta có thể sử dụng ma trận 2D để lưu thông tin vùng.

- Mỗi ô ma trận lưu chỉ số vùng mà nó thuộc về hoặc -1 nếu không thuộc vùng nào.
- Truy cập ma trận trực tiếp để xác định Hunter thuộc vùng nào.

Phương pháp thuật toán:

- Đánh dấu vùng trên ma trận 2D.
- Dùng trực tiếp $O(1)$ để truy xuất vùng cho từng Hunter.

Lý do chọn phương pháp:

- Tránh phải duyệt tất cả các vùng cho mỗi Hunter (giảm từ $O(n \cdot m)$ xuống $O(1)$ cho mỗi Hunter).

- Phù hợp khi $W \cdot H \leq 10^6$, bộ nhớ vẫn trong giới hạn.

Phân tích độ phức tạp:

- Đánh dấu ma trận: $O(\sum_i (A_i - X_i)(B_i - Y_i)) \leq O(W \cdot H)$
- Xử lý Hunter: $O(m)$
- Độ phức tạp thời gian: $O(W \cdot H + m)$

- Bộ nhớ: $O(W \cdot H)$

Code Python:

```

1 def subtask2(W, H, n, m, regions, hunters):
2     grid = [[-1]*H for _ in range(W)]
3     for i, (X, Y, A, B) in enumerate(regions):
4         for x in range(X, A):
5             for y in range(Y, B):
6                 grid[x][y] = i
7
8     ans = [0] * (n + 1)
9     for cx, cy, t in hunters:
10        if 0 <= cx < W and 0 <= cy < H:
11            idx = grid[cx][cy]
12            if idx == -1:
13                ans[n] += t
14            else:
15                ans[idx] += t
16        else:
17            ans[n] += t
18    return ans

```

3.3 Subtask 3: Không giới hạn

Hướng tiếp cận:

- Với $n, m \leq 10^5$ và $W, H \leq 10^9$, không thể dùng brute force hoặc ma trận.
- Sử dụng Sweep Line theo trục X:
 - Sắp xếp các sự kiện: mở vùng, Hunter, đóng vùng theo X.
 - Active set lưu các vùng đang mở, được sắp xếp theo tọa độ Y dưới.
 - Khi gặp Hunter, dùng binary search trên active set để xác định vùng nhỏ nhất chứa Hunter.
 - Nếu không thuộc vùng nào, cộng Aura vào phần không theo dõi.

Phương pháp thuật toán:

- Sweep Line kết hợp Binary Search.
- Đảm bảo xử lý dữ liệu lớn hiệu quả.

Lý do chọn phương pháp:

- Mỗi sự kiện xử lý $O(\log n)$ thay vì $O(n)$. - Tổng số sự kiện: $2n + m \leq 3 \cdot 10^5$, đủ nhanh.

Phân tích độ phức tạp:

- Số sự kiện: $2n + m$
- Xử lý mỗi sự kiện: $O(\log n)$ (thêm/xóa vào active set, tìm vùng)
- Tổng thời gian: $O((n + m) \log n)$
- Bộ nhớ: $O(n)$

Code Python:

```
1 import bisect
2
3 def subtask3(W, H, n, m, regions, hunters):
4     events = []
5     for i, (X, Y, A, B) in enumerate(regions):
6         events.append((X, 0, i))
7         events.append((A, 2, i))
8     for j, (C, R, t) in enumerate(hunters):
9         events.append((C, 1, j))
10
11     events.sort(key=lambda x: (x[0], x[1]))
12     active_y = []
13     ans = [0]*(n+1)
14
15     for x, typ, idx in events:
16         if typ == 0: # m v ng
17             X, Y, A, B = regions[idx]
18             bisect.insort(active_y, (Y, idx))
19         elif typ == 2:
20             X, Y, A, B = regions[idx]
21             pos = bisect.bisect_left(active_y, (Y, idx))
22             if pos < len(active_y) and active_y[pos][1] == idx:
23                 active_y.pop(pos)
24         else:
25             C, R, t = hunters[idx]
26             pos = bisect.bisect_right(active_y, (R, 10**15)) - 1
27             if pos >= 0:
28                 Y0, rid = active_y[pos]
```

```

29         X, Y, A, B = regions[rid]
30         if Y <= R < B and X <= C < A:
31             ans[rid] += t
32         else:
33             ans[n] += t
34     else:
35         ans[n] += t
36     return ans

```

Nhận xét:

- Phương pháp này mở rộng tốt cho dữ liệu lớn.
- Sắp xếp và sweep line giúp tránh phải kiểm tra toàn bộ vùng cho từng Hunter, giảm đáng kể thời gian thực thi.
- Binary search trong active set giúp xác định vùng phù hợp một cách hiệu quả.